ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

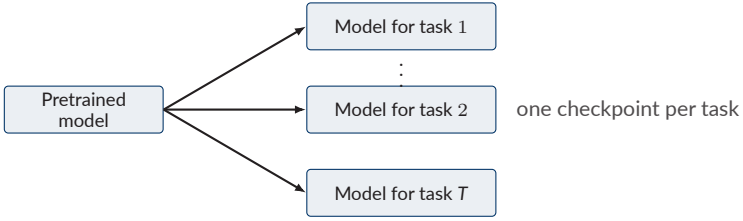
MERGING 5 GLUE NLP TASK USING WEIGHT-ENSEMBLING MIXTURE OF EXPERTS

Few-shot router training across five GLUE tasks

Group 1
Nguyễn Bá Anh – 20261218M
Nguyễn Thị Nhã Linh – 20261262M
Instructor: Nguyễn Kiêm Hiếu

ONE LOVE. ONE FUTURE.

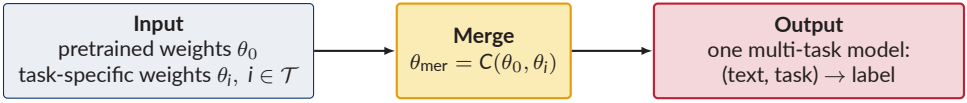
One model per task does not scale



- **Management:** every new task adds another full model to train, store and serve.
- **Isolation:** each model knows only its own task, so nothing it learned is shared.
- **Redundancy:** the models start from the same weights, yet that common part is stored again each time.

Next: can these separate models become a single model?

Problem statement



Find the merge function C so that the merged model:

- integrates task knowledge **dynamically**, depending on the input;
- stays close to the quality of the task-specific models;
- needs fewer stored parameters than keeping all the models.

Next: why is a simple merge not enough?

Why this problem: static merging falls short

Why it is worth solving

- Real systems serve many related NLP tasks.
- A shared model makes knowledge reusable across tasks.
- Merging needs no joint retraining from scratch.

Why it is hard

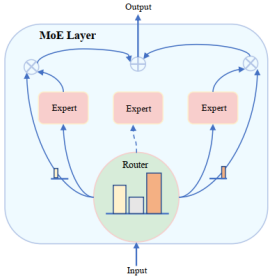
- **Interference:** updates from different tasks can conflict in the same parameters.
- **Sign disagreement:** models can pull one weight in opposite directions.
- **Static weights:** one fixed result cannot be the best for every task at once.

Gap

The merged weights should change with the input, not stay fixed for all inputs.

Next: how can weights depend on the input?

Idea: Mixture of Experts



Mixture of Experts layer[6]

$$\mathrm{MoE}(x) = C\left(\{G_i(x) \cdot E_i(x)\}_{i=1}^{n}\right)$$

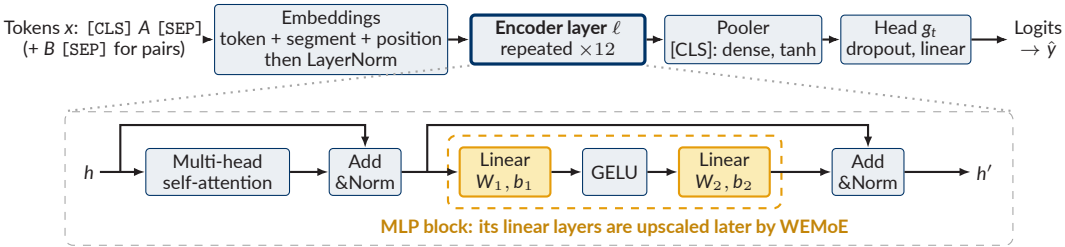
- Router G decides how much each expert counts for **this input**.[1]
- That is exactly the dynamic behaviour the problem asks for.

From outputs to weights

Running every expert is what we want to avoid. **WEMoE** mixes experts in **weight space**: the expert is a task vector, and one forward pass is enough.[2]

Next: which backbone do we merge?

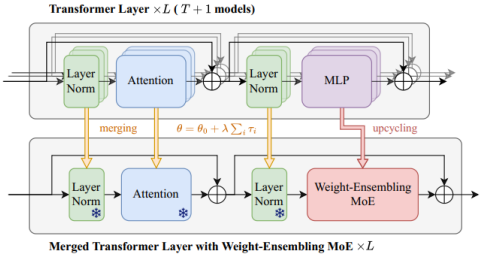
BERT encoder: the shared backbone



- `bert-base-uncased`: 12 encoder layers, 109.5M parameters.[3] All experts share this structure, so weights can be subtracted and added position by position.
- Self-attention gives each token a state that reflects its whole context. The [CLS] state passes through the pooler to the head of task t.
- CoLA and SST-2 use a single sentence; MRPC, RTE and QNLI use a sentence pair.

Next: how is this structure merged?

WEMoE architecture: two kinds of merging

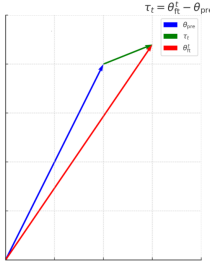


Top: the fine-tuned and pretrained models. Bottom: the merged layer.[2]

- **Static:** weights outside the MLP are merged once with Task Arithmetic. In this project that also covers embeddings and pooler.
- **Dynamic:** each MLP is upscaled to a Weight-Ensembling MoE module, so its weights depend on the input.
- Frozen parts carry the snowflake mark; only the router will be trained.

Next: first the static part, then the module.

Task Arithmetic: static merge with task vectors



Task vector: what fine-tuning changed for task i[4]

$$\tau_i = \theta_i - \theta_0 \qquad (\theta_{\text{pre}} = \theta_0,\ \ \theta_{\text{ft}}^i = \theta_i)$$

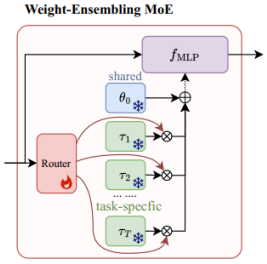
Task Arithmetic: add the vectors back to the base

$$\theta = \theta_0 + \lambda \sum_{i=1}^{n} \tau_i$$

- λ controls how strongly the task vectors contribute; it is chosen on dev.
- Simple and compact, but the result is the same for every input.

Next: make the coefficients depend on the input, in the MLPs.

Router and WEMoE module: input-conditioned MLP



WEMoE module; here $T = 5$[2]

$$w^\ell(x) = \frac{1}{N}\sum_{j=1}^{N} r^\ell(h_j) \in \mathbb{R}^5$$

$$W^\ell(x) = W_0^\ell + \sum_{e=1}^{5} w_e^\ell(x)\,\tau_e^\ell$$

$$r^\ell(h) = \mathrm{fc}_2\big(\mathrm{ReLU}(\mathrm{fc}_1(h))\big)$$

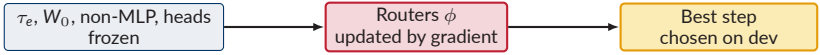
- One router per encoder layer; the mean runs over non-padding tokens; both MLP linear layers share w^ℓ.
- No softmax: a weight can exceed 1 or be negative, so a router can amplify or subtract a task vector.
- The router's bias starts at λ, so the model starts close to Task Arithmetic.

Objective: few-shot fine-tuning (FSFT)

Cross-entropy on a small labelled set, summed over the tasks:

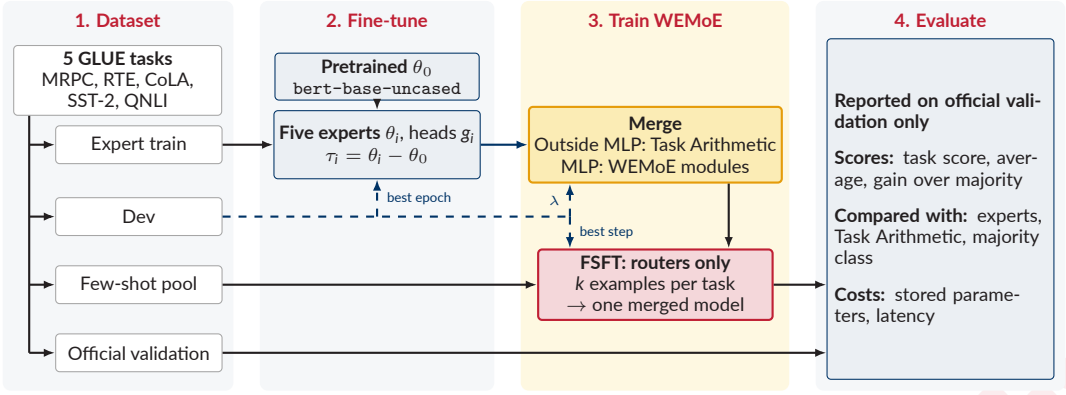
$$\min_{\phi} \sum_{t=1}^{5} \frac{1}{|B_t|} \sum_{(x,y)\in B_t} -\log p_\phi(y \mid x, t), \qquad B_t \subset D_t^{(k)}, \;\; |D_t^{(k)}| = k$$

- ϕ: router parameters, **the only trainable weights** (7.1M).
- $D_t^{(k)}$: k labelled examples of task t, which the experts have never seen.
- p_ϕ: softmax over the logits of head g_t.



Next: how does the whole pipeline run?

Implementation flow: from dataset to evaluation



Expert weights stay frozen after stage 2; only the routers learn in stage 3. The task id selects the head.

Data: five binary GLUE tasks

Task	Decides whether…	Input fields	Labels	Metric
MRPC	two sentences are paraphrases	sentence1, sentence2	not_equivalent / equivalent	acc + F1
RTE	a sentence entails another	sentence1, sentence2	entailment / not_entailment	accuracy
CoLA	a sentence is grammatical	sentence	unacceptable / acceptable	MCC
SST-2	a sentence is positive or negative	sentence	negative / positive	accuracy
QNLI	a sentence answers a question	question, sentence	entailment / not_entailment	accuracy

- GLUE benchmark.[5] Inputs are tokenised to a maximum length of 128.
- **Task score:** MRPC uses the mean of accuracy and F1. Scores are $\times 100$.
- **Average score:** unweighted mean of the five task scores.

Data split: keep each stage's data apart

Task	Official train	Few-shot pool	Dev	Expert train	Official validation
MRPC	3,668	256	367	3,045	408
RTE	2,490	256	249	1,985	277
CoLA	8,551	256	855	7,440	1,043
SST-2	67,349	256	1,000	66,093	872
QNLI	104,743	256	1,000	103,487	5,463
Used for		**FSFT**	**Selection**	**Experts**	**Reporting**

- Official train is split once, with seed 42. Dev is 10% of train, at most 1,000 examples.
- Experts never see the few-shot pool. k examples per task are drawn from it.
- Every choice (expert epoch, λ, FSFT step) is made on dev. Official validation is used for reported scores only.

Stage 2: fine-tuning the five experts

Hyperparameters:

Setting	MRPC	RTE	CoLA	SST-2	QNLI
Learning rate	2e-5	2e-5	2e-5	2e-5	2e-5
Weight decay	1e-2	1e-2	1e-2	1e-2	1e-2
Scheduler type	linear	linear	linear	linear	linear
Warmup ratio	0.10	0.10	0.10	0.10	0.10
Gradient clip norm	1	1	1	1	1
Batch size / epochs	32 / 3	32 / 3	32 / 3	32 / 3	32 / 3

Table: Fine-tuning settings of `bert-base-uncased`: one recipe for all five GLUE tasks.

Results:

MRPC (Acc/F1)	RTE (Acc)	CoLA (MCC)	SST-2 (Acc)	QNLI (Acc)
82.1 / 88.1	63.2	59.1	92.2	91.2

Table: Expert scores on official validation ($\times 100$); average score 78.2. Best epoch on dev: 2, 3, 3, 2, 3.

Stage 3: merging and few-shot router training

Hyperparameters:

Setting	Full, k=16	Full, k=64	Full, k=256	Intermediate, k=256	Input-indep., k=256
Upscaled MLP layers	both	both	both	first only	both
Trainable parameters	7.1M	7.1M	7.1M	7.1M	60
Learning rate	1e-4	1e-4	1e-4	1e-4	1e-3

Table: All runs: AdamW, batch 16 per task, 500 steps, linear schedule, warmup 0.10, no weight decay, no gradient clipping. Task Arithmetic: $\lambda = 0.3$, best on dev from $\{0.1, \ldots, 1.0\}$.

Results:

Metric	Task Arith.	Full, k=16	Full, k=64	Full, k=256	Interm., k=256	Input-indep., k=256
Average score	57.4	60.8 ± 2.7	64.6 ± 3.7	$\mathbf{69.9 \pm 0.1}$	69.1 ± 0.1	58.4 ± 0.0
Gain over majority	18.4	25.5	46.7	**59.1**	54.8	20.0
Best step of 500	–	50, 0, 150	100, 250, 100	250, 200, 200	500, 400, 400	50, 50, 50

Table: Official validation ($\times 100$), mean $\pm$ std over seeds 0, 1, 2 (a seed sets the router initialisation and the few-shot sample). Reference points: majority class 45.8, experts 78.2. Gain $= \frac{\text{score} - \text{majority}}{\text{expert} - \text{majority}}$, averaged over tasks.

Takeaway: fixed coefficients barely move the score; input-conditioned routers close over half the gap. (Input-independent: one learning rate tried, dev score nearly flat.)

Results by task: the gain is not uniform

Model	MRPC (Acc/F1)	RTE (Acc)	CoLA (MCC)	SST-2 (Acc)	QNLI (Acc)
Majority class	68.4 / 81.2	52.7	0.0	50.9	50.5
Task Arithmetic	68.6 / 81.3	47.3	16.9	69.8	77.9
WEMoE full, k=256	76.3±4.5 / 83.9±2.8	**52.0 ± 4.4**	42.6 ± 0.9	88.0 ± 0.5	86.7 ± 0.7
WEMoE intermediate, k=256	**66.5±0.7 / 76.6±1.0**	59.0 ± 0.9	41.7 ± 0.9	86.7 ± 0.2	86.5 ± 0.2
Single-task experts	82.1 / 88.1	63.2	59.1	92.2	91.2

Table: Per-task scores on official validation ($\times 100$), mean ± std over seeds 0, 1, 2.

- **Not solved everywhere:** full MLP does not reliably beat majority on RTE; intermediate-only falls below it on MRPC.
- **Budget matters:** small k varies strongly across seeds (CoLA MCC std 14.1 at $k = 16$, 17.8 at $k = 64$); $k = 256$ is stable on the average score.

Trade-offs and conclusion

Deployment	Stored params	vs. 5 models	Avg score	ms/example (batch 32)
5 single-task models	547.4M	100.0%	78.2	1.61*
Task Arithmetic	109.5M	20.0%	57.4	1.63
WEMoE, full MLP	400.0M	73.1%	69.9	7.31
WEMoE, intermediate-only	258.4M	47.2%	69.1	4.69

Table: Latency on one T4, random 128-token inputs, fp16. *Times one expert: each input goes to its own expert by task id.

- **Answer:** input-conditioned merging with few-shot routers turns separate models into one, and clearly beats the static merge on average.
- **Cost:** it stays below the experts and is slower per example; the storage saving is largest for intermediate-only.
- **Limits:** RTE and MRPC remain weak; model selection uses more labels than FSFT trains on; experts use one seed; RTE and MRPC have small validation sets.
- This is one full run on five tasks, not a general guarantee.

Backup: router details

- Two linear layers with ReLU, applied per token; outputs are averaged over non-padding tokens.
- Initial weights are drawn with std 0.01 and the final bias is set to λ.
- Input-independent variant: one learned coefficient per expert per layer, 60 numbers in total.
- Experts do not run at inference: only the task vectors and heads are stored.

References

[1] Shazeer et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. arXiv:1701.06538, 2017.

[2] Tang et al. Merging Multi-Task Models via Weight-Ensembling Mixture of Experts. arXiv:2402.00433, 2024.

[3] Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805, 2019.

[4] Ilharco et al. Editing Models with Task Arithmetic. arXiv:2212.04089, 2023.

[5] Wang et al. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. arXiv:1804.07461, 2018.

[6] Mu and Lin. A Comprehensive Survey of Mixture-of-Experts: Algorithms, Theory, and Applications. arXiv:2503.07137, 2025.